# 11_21_aprile

## Scheduling su sistemi multiprocessore

(Con multiprocessore intendiamo singola CPU con più core fisici)

### Approcci di scheduling

Abbiamo diversi modi per fare scheduling su questi tipi di dispositivi:

- *multi-elaborazione asimmetrica*: uno dei processori assume il ruolo di *master-server* e decide quale processore esegue un certo processo (se abbiamo 8 core 1 fa girare il software dello scheduler e fa solo quello e gli altri tutti gli altri processi)
  - *PRO*: Si azzerano i casi di race condition
  - *CONTRO*: Quando un singolo core deve gestire le code per centinaia di core, questo rallenta e li rallenta tutti
- *multi-elaborazione simmetrica*: coda unificata dei processi pronti o code separate per ogni processore/core
  - *PRO***
    - Nessun collo di bottiglia: Non essendoci un processore "master", il carico di lavoro dello scheduling è distribuito.
    - Tutti i core partecipano attivamente all'esecuzione dei processi (sia utente che di sistema), senza sprecare risorse dedicando un'intera unità solo alla gestione delle code.
  - *CONTRO*:
    - Problemi di race condition
    - *Cache Affinity*: Se un processo viene sospeso e poi ripreso da un *core diverso*, i dati che aveva salvato nella memoria cache della CPU originaria diventano inutili. Ricaricare i dati dalla memoria centrale alla cache del nuovo core rallenta notevolmente l'esecuzione.

E' quindi importante che un processo riprenda e finisca la sua esecuzione nello stesso core in cui è iniziata, questo per via di come funziona la cache e il prefetching con linee di cache perché se un processo riprende l'esecuzione in un core diverso da quello di partenza si verificano ritardi dovuti all'accesso alla RAM anziché alla cache, ovviamente questo non può sempre accadere, delle volte si avranno core diversi per lo stesso processo. Infatti...

### Politiche di scheduling

- **Predilezione Debole**: Con questo approccio, il sistema operativo cerca di fare del suo meglio per mantenere un processo sullo stesso processore su cui stava girando in precedenza, proprio per sfruttare un possibile cache hit

- **Predilezione Forte**: In questo caso, vengono impostati dei vincoli stretti. Il sistema operativo (spesso su richiesta esplicita del programmatore o dell'amministratore di sistema) garantisce che un determinato processo venga eseguito **esclusivamente** su uno specifico processore o su un gruppo limitato di processori scelti.

### Bilanciamento del carico vs. Predilezione

- Il **bilanciamento del carico** spinge per muovere i processi da un core all'altro (migrazione) per assicurarsi che nessun processore rimanga con le mani in mano.
- La **predilezione** spinge per tenere i processi fermi esattamente dove si trovano per massimizzare l'efficienza della memoria.
  Queste due politiche "remano contro" l'una all'altra. I sistemi operativi moderni (come Linux o FreeBSD citati nella slide) implementano algoritmi molto complessi per trovare un compromesso.

**Migrazione guidata (push)**
**Migrazione spontanea (pull)**

## Cosa usano i nostri sistemi operativi

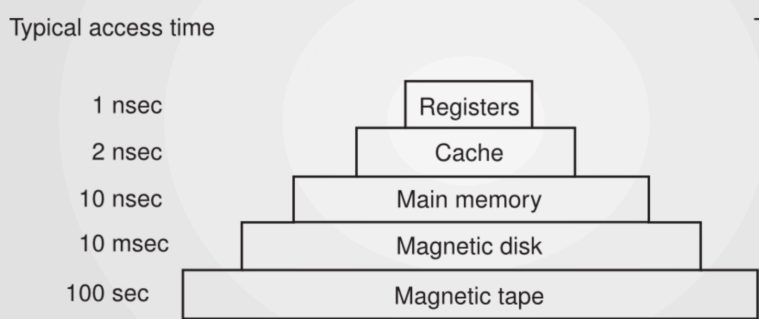
- **Windows**:
  - scheduler basato su code di priorità;
  - euristiche per migliorare il servizio dei processi interattivi e in particolare di foreground;
  - euristiche per evitare il problema dell'inversione di priorità.
- **Linux**:
  - scheduling basato su task (generalizzazione di processi e thread);
  - Completely Fair Scheduler (CFS): moderno scheduler garantito;
- **MacOS**:
  - Mach scheduler basato su code di priorità con euristiche.

---

Siamo passati al file SO-03.pdf

## Gestione della memoria centrale

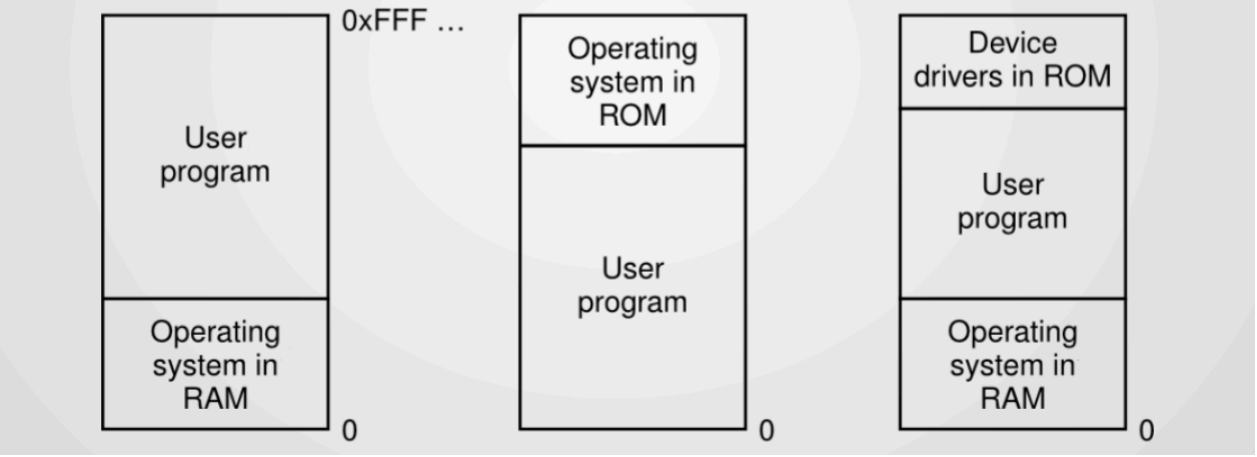
Nelle memorie abbiamo una certa gerarchia (dalla più veloce alla più lenta), la RAM è il livello più basso che la CPU può direttamente usare:




In generale la memoria viene gestita usando degli indirizzi. Nel tempo la gestione degli indirizzi è cambiata:

- **Senza astrazione**: i programmi utilizzano direttamente gli indirizzi fisici, non venivano utilizzate strutture dati complicate quindi questa gestione andava bene, esistono diversi modelli, in base a come erano fisicamente costruiti di dispositivi (anni 60- primi anni 80)

  Usare gli indirizzi fisici significa scrivere direttamente nella cella numerata della memoria, se in un programma c'è un'istruzione che dice "salva questo numero all'indirizzo 2000", il processore va *letteralmente* nella cella di memoria numero 2000 e ci scrive dentro




- **Multiprogrammazione senza astrazione**: in questo scenario, più programmi risiedono contemporaneamente nella memoria RAM. Gli indirizzi sono quelli fisici. Questo approccio introduce due problemi critici che devono essere risolti per garantire il corretto funzionamento del sistema:
  1. **Rilocazione (Relocation):** Quando un programmatore scrive un codice, non può sapere in anticipo in quale zona di memoria verrà caricato il programma. Se il codice contiene riferimenti a indirizzi assoluti (es. "vai all'indirizzo 100"), e il programma viene caricato a partire dall'indirizzo 2000, il software fallirà.
     - **Soluzione statica:** Il *loader* (il componente che carica il programma) modifica gli indirizzi nel codice al momento del caricamento, sommandovi l'indirizzo di base dove il programma è stato depositato.
  2. **Protezione (Protection):** In un sistema dove più processi convivono senza barriere astratte, c'è il rischio che un processo possa scrivere nell'area di memoria di un altro processo o in quella del Sistema Operativo, causano crash o furto di dati.

    - **Soluzione (Lock & Key):** Si divide la memoria in blocchi e a ciascuno si assegna una "chiave" (un codice numerico). Il processo può accedere solo ai blocchi la cui chiave corrisponde a quella memorizzata nel suo registro di stato
- **Astrazione dello spazio di indirizzi**: per ogni processo viene specificato lo spazio di indirizzi, utilizzabile da un processo. In pratica viene fatta un'allocazione dinamica che permette ad ogni indirizzo di partire dall'indirizzo 0 rispetto al suo spazio, questo viene fatto usando:
    - *registro base*: contiene l'indirizzo fisico di partenza dove il processo è effettivamente caricato in RAM. Ogni indirizzo generato dal programma (indirizzo logico) viene sommato al valore di questo registro per ottenere l'indirizzo fisico reale. Nei sistemi moderni, questo compito di traduzione da indirizzo logico a fisico è delegato a un'unità dedicata chiamata MMU.
    - *registro limite*: definisce la dimensione massima del processo. Serve a garantire che il processo non acceda a memoria fuori dal suo intervallo consentito. La CPU verifica in tempo reale che l'indirizzo logico sia inferiore al valore nel registro limite. Se il controllo fallisce, viene generata una TRAP che passa il controllo al SO, che terminerà inevitabilmente il processo in questione. Questo modello, utilizzato in macchine storiche come il CDC 6600 o l'Intel 8088, rappresenta il primo vero passo verso l'isolamento dei processi.

## Swapping

Il livello di multiprogrammazione è limitato dalla dimensione della memoria centrale, da qui nascono diverse soluzioni, la prima di queste è lo *swapping* (non è l'area di swap famosa di win). Se viene richiesto di creare un nuovo processo ma la memoria centrale è piena, lo swapper (*scheduler di medio termine*) decide quale processo inserire nel disco (SSD o Hard Disk), in modo da liberare la memoria e poter creare il nuovo processo.

> **Tip**
>
> Quando la RAM è piena e il sistema ha bisogno di eseguire un nuovo programma, lo swapper seleziona un processo in memoria che al momento non sta facendo nulla e compie un'operazione di **swap out**, parcheggiandolo temporaneamente su un disco capiente chiamato *backing store*. Lo spazio appena liberato viene subito riutilizzato per un'operazione di **swap in**, prelevando un nuovo processo dal disco e caricandolo nella memoria centrale affinché la CPU possa eseguirlo.

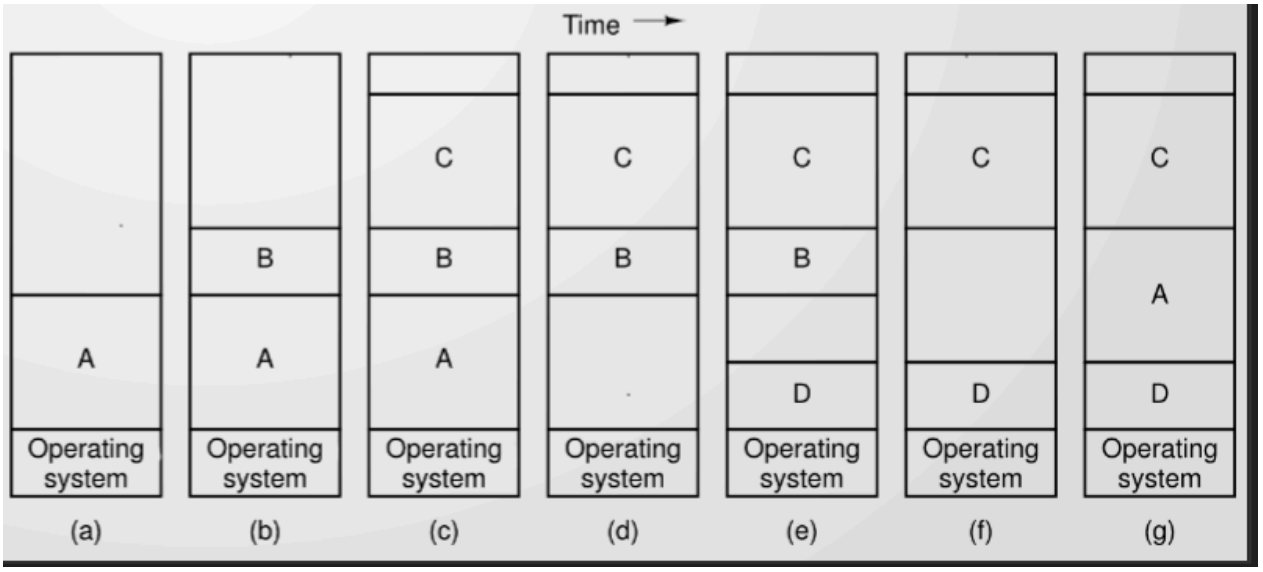
Questa strategia crea delle problematiche, come i problemi delle operazioni di input/output: cosa succede se trasferiamo su disco un processo che stava aspettando la scrittura di un dato da parte dell'hardware? Il rischio è che la periferica finisca per scrivere quel dato nello stesso punto della RAM, ma che ora appartiene a un altro

processo, corrompendone i dati. Per evitarlo, il sistema operativo deve ancorare in memoria i programmi impegnati in queste operazioni.

Oltre a questo, c'è il problema di come allocare lo spazio fisico. Sebbene si possa dividere la memoria in blocchi a **dimensione fissa**, spesso si usa una **dimensione dinamica** che dà a ogni processo l'esatta memoria richiesta.

Questo però genera il fenomeno della **frammentazione**, dalla quale distinguiamo due casi:

- *interna*: frammentazione interna al blocco assegnato ad un processo
- *esterna*: Man mano che i processi (A, B, C, D) vengono caricati, eseguiti e scaricati, la memoria si riempie di piccoli "buchi" inutilizzati. Col tempo, si arriva a un paradosso: la somma di tutti questi buchi potrebbe bastare per caricare un nuovo programma, ma siccome lo spazio non è contiguo, il sistema non sa dove metterlo.




Quando la frammentazione esterna diventa critica entra in gioco la **memory compaction**: il sistema operativo entra in azione riorganizzando fisicamente l'intera RAM: sposta e "schiaccia" tutti i processi verso un'estremità della memoria per fondere tutti i piccoli buchi in un unico grande spazio libero. È una manovra risolutiva, ma molto costosa in termini di prestazioni, poiché blocca temporaneamente l'esecuzione di tutto il resto

## Gestione dell'allocazione

Dopo che il SO ha deciso come gestire la memoria, ha bisogno di uno strumento pratico per farlo. Deve sapere in ogni istante, con precisione, quali parti della RAM sono libere e quali sono occupate. Qui distinguiamo due metodologie:

- La bitmap:
- Lista dei blocchi liberi e occupati: